

GitHub Integration - Complete Test Guide

Objective

Test end-to-end GitHub integration dan pembuatan developer profile untuk memastikan flow utama aplikasi CoreDev Zero berjalan dengan benar.

Test Scenarios

1. GitHub Verification (Mock Mode)

Test Mock Data

- **URL:** http://localhost:3000/github-test
- **Test Users:**
 - **testdev1** - Verified developer (high trust score: 220)
 - **testdev2** - Verified developer (medium trust score: 180)
 - **newdev** - Unverified developer (low trust score: 110)
 - **randomuser** - Should return error

Expected Results

-  Mock data returns proper GitHubVerificationData structure
-  Trust score calculation works
-  Verification status based on criteria (repos >= 5, age >= 6 months, contributions >= 100)
-  Top languages are populated
-  Error handling for unknown users

2. Developer Profile Creation Flow

Step-by-Step Test

1. Connect Wallet

- Go to http://localhost:3000/dashboard
- Connect to Core DAO Testnet (Chain ID: 1114)
- Ensure wallet has some tCORE2 for gas

2. GitHub Verification

- Should see GitHub verification form first
- Test with username: **testdev1**
- Verify GitHub data is fetched and displayed
- Click "Continue with GitHub Data"

3. Profile Creation

- Should see profile form with pre-filled GitHub handle
- GitHub data summary should be displayed
- Skills should be suggested from top languages
- Fill in bio (optional)
- Click "Create Profile"

4. Transaction Flow

- Transaction should be submitted to MarketFactory contract
- Wait for confirmation
- Success message should appear
- Profile should be created on-chain

Expected Results

-  GitHub verification completes successfully
-  Profile form shows GitHub data
-  Transaction submits to correct contract (MarketFactory)
-  Profile is created with GitHub verification data
-  Trust score is calculated and stored
-  Explorer link works (<https://scan.test2.btcs.network>)

3. Real GitHub API Test (Optional)

Setup for Real API

1. Create GitHub OAuth App (for production)
2. Set environment variables:

```
GITHUB_CLIENT_ID=your_client_id
GITHUB_CLIENT_SECRET=your_client_secret
```

3. Update `useMockData={false}` in GitHubVerification

Test with Real Data

- Test with actual GitHub usernames
- Verify rate limiting works
- Test error handling for private/non-existent users

Technical Components Tested

GitHubIntegrationService

- Mock data generation
- Real GitHub API calls
- Trust score calculation

- Account verification logic
- Error handling

GitHubVerification Component

- UI for username input
- Data display and visualization
- Mock/real API toggle
- Verification requirements display

CreateProfileForm Component

- Two-step flow (GitHub → Profile)
- Data pre-filling from GitHub
- Smart contract integration
- Transaction handling
- Error states

DeveloperDashboard Component

- Profile existence check
- Integration with CreateProfileForm
- Profile data display after creation

Common Issues & Solutions

1. MetaMask Network Issues

Problem: Not connected to Core DAO Testnet

Solution:

- Add Core DAO Testnet manually
- RPC: <https://rpc.test2.btcs.network>
- Chain ID: 1114
- Currency: tCORE2

2. Contract Address Issues

Problem: Contract not found errors

Solution: Ensure deployed addresses in contracts.ts match hardhat deployment

3. GitHub API Rate Limiting

Problem: API calls fail after many requests

Solution: Use mock data for development, implement proper error handling

4. Transaction Failures

Problem: Profile creation fails

Solution:

- Check gas limit and gas price
- Verify contract ABI is correct
- Ensure wallet has enough tCORE2



Success Criteria

Phase 1: GitHub Integration

- Mock GitHub data works
- Real GitHub API integration (structure ready)
- Trust score calculation
- Verification requirements check
- UI components working

Phase 2: Profile Creation

- Two-step creation flow
- GitHub data integration
- Smart contract deployment
- Transaction handling
- Error handling and validation

Phase 3: Dashboard Integration

- Profile status detection
- Seamless flow from no profile → creation → dashboard
- GitHub verification data display
- Trust score display



Next Steps After Success

1. Market Creation

- Test creating lending markets
- Verify borrowing limits based on trust score
- Test market parameters

2. Lending/Borrowing

- Test loan requests
- Test loan approvals
- Test repayment flow

3. NFT Integration

- Test loan position NFT creation
- Test marketplace listings
- Test NFT transfers



Test URLs

- Dashboard: <http://localhost:3000/dashboard>
- GitHub Test: <http://localhost:3000/github-test>
- Actions (Debug): <http://localhost:3000/actions>
- Markets: <http://localhost:3000/markets>
- Marketplace: <http://localhost:3000/marketplace>



Expected User Journey

1. User connects wallet → Dashboard
2. No profile detected → GitHub verification form
3. User enters GitHub username → Verification success
4. User fills profile form → Transaction submitted
5. Profile created → Dashboard shows profile info
6. User can now create markets and participate in ecosystem

Status:  GitHub Integration Complete - Ready for End-to-End Testing

Next Phase: Market Creation & Lending Flow Testing